

Today's Agenda :-

→ Producer Consumer

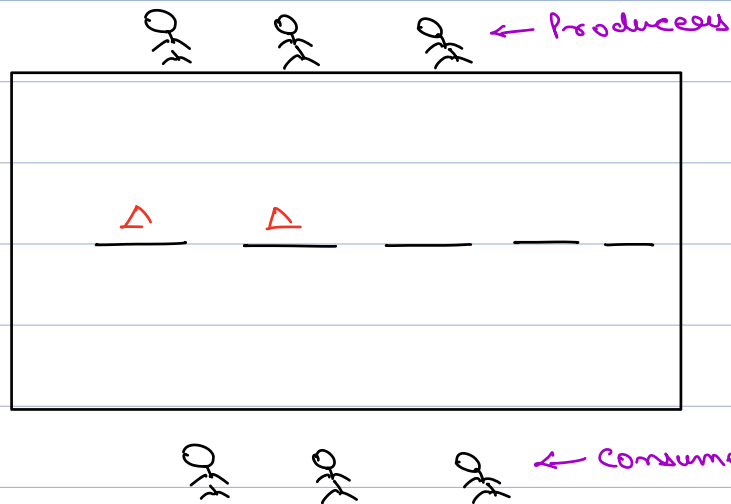


Semaphores

→ 9 LeetCode Problems

→ Atomic Datatypes.

Revise Producer Consumer
from last
session



V_1 :- Sync issues,
 V_2 :- Synchronized block

↓
 1 thread was allowed at 1 time

max size = 5,

① $\# \text{Producers} + \# \text{Consumers} \leq 5$

items	Prod	cons
<u>2 items</u>	3 producers	2 consumers
3	2	3
0	5	0
1	4	1
4	1	4

② Producers $\xrightarrow{\text{Signal}}$ consumers

locks + Count

permit

permit

1 1 1 1 1

Producers $\Rightarrow$ ~~5~~ ~~4~~ ~~7~~ ~~2~~ ~~5~~ ~~1~~ 21

Consumers $\Rightarrow$ ~~0~~ ~~8~~ ~~2~~ ~~0~~ ~~8~~ 4

Producer --- ;
produce the item

Consumer -- ; ✓

Consume ;

Consumer ++

producer ++ ; ✓

Δ Δ Δ Δ Δ

Semaphores

mutex + count

more than 1 thread in CS.

signalling.

Semaphores `prod = new Semaphore(5);`

Semaphores `cons = new Semaphore(1);`

`prod = 3` ~~3~~ ~~2~~

`cons = 2` ~~2~~ ~~1~~

`prod.acquire()` { }

`cons.acquire()`; { }

`cons.release()`

`prod.release()`;

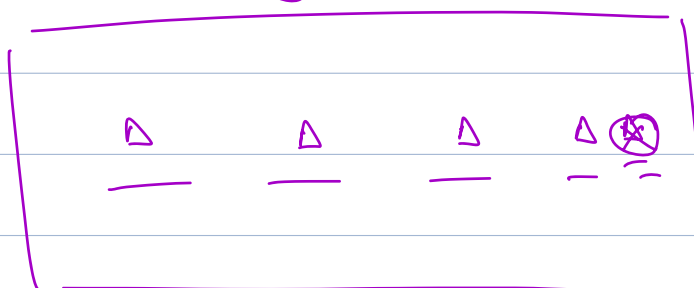


```
public void removeItem() { 1 usage new *
    System.out.println("Consumer consuming time, current size " + this.items.size());
    this.items.remove(index, this.items.size() - 1);
    System.out.println("Consumer item, current size " + this.items.size());
}
```

4

Producer →

5

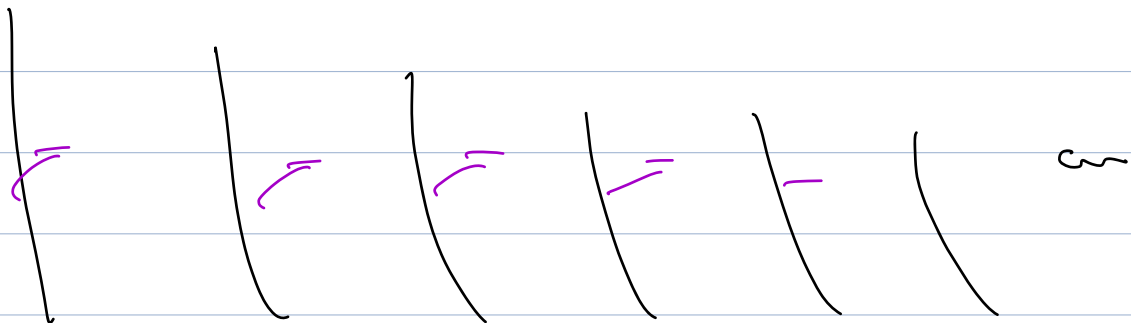
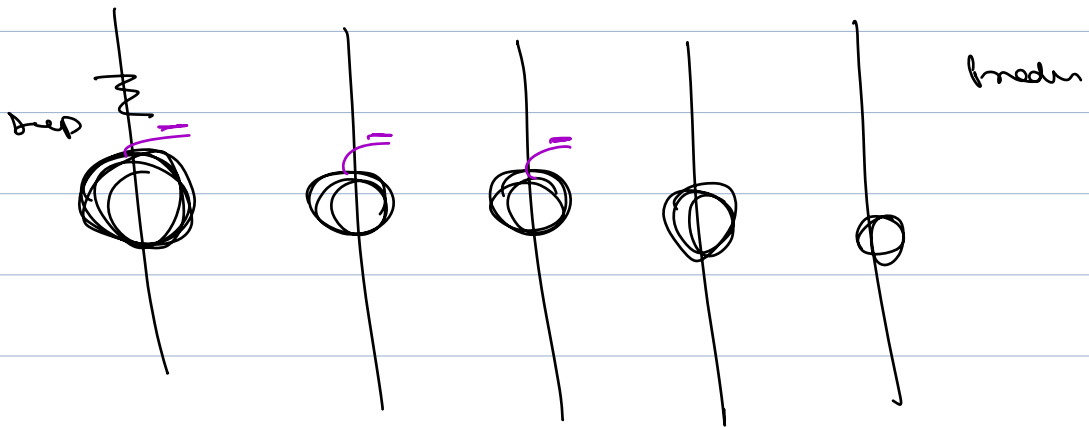


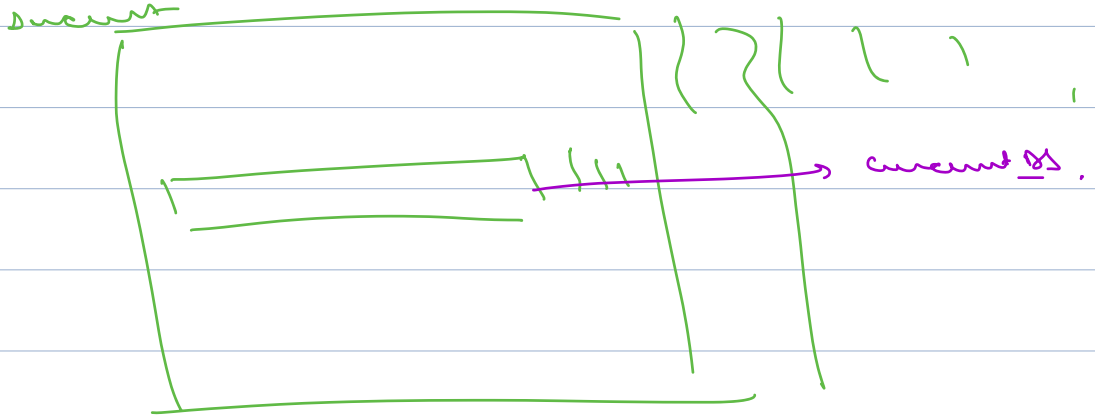
Cons = 5

Sum
Consumer



Sum
Producer
5





val += i

10:29pm - 10:39pm

zero = x x x 0
even = x x 0
odd = x x 0

{ i = 1, 3

0 1 0 2

{ i = 2

```
// printNumber.accept(x) outputs "x", where x is an integer.
public void zero(IntConsumer printNumber) throws InterruptedException {
    for(int i=1; i<=n; i++){
        zero.acquire();
        printNumber.accept(0);
        if(i%2==0){
            even.release();
        }else{
            odd.release();
        }
    }
}
```

```
public void even(IntConsumer printNumber) throws InterruptedException {
    for(int i=2; i<=n; i+=2){
        even.acquire();
        printNumber.accept(i);
        zero.release();
    }
}
```

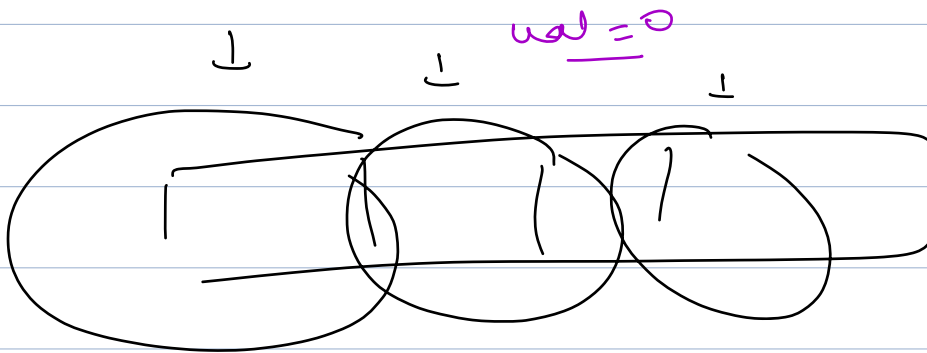
{ i = 2

```
public void odd(IntConsumer printNumber) throws InterruptedException {
    for(int i=1; i<=n; i+=2){
        odd.acquire();
        printNumber.accept(i);
        zero.release();
    }
}
```

Compare and Swap

$val += i$
 $t \leftarrow val$
 $t \rightarrow t + i \rightarrow 1$
 $t \rightarrow val$
1

$val -= i$
 $t \leftarrow val \rightarrow n$
 $t \rightarrow t - i$
 $t \rightarrow val$
 $if val == n ?$
 $\downarrow$
3



$\downarrow \rightarrow \underline{\text{lock 1}}$
 $\underline{\text{lock 2}} \leftarrow \downarrow$
lock 1. lock (); -
 lock 2. lock ();
 $\rightarrow \text{lock 2. lock ()}$
 lock 1. lock ()

Deadlock

3 ways to handle deadlock

① Deadlock Avoidance

Multiple locks please ensure

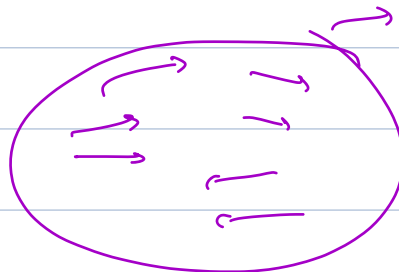
lock 1. try new lock (-) taking them in same order
 $\rightarrow$
lock 1. lock (); - $\rightarrow \underline{\text{lock 1}}$
 lock 2. lock (); $\rightarrow \underline{\text{lock 2}}$
 $\rightarrow \underline{\text{lock 1. lock ()}}$
 $\rightarrow \underline{\text{lock 2. lock ()}}$

-
 -
 -

② Deadlock recovery, $\rightarrow$ Restart Process

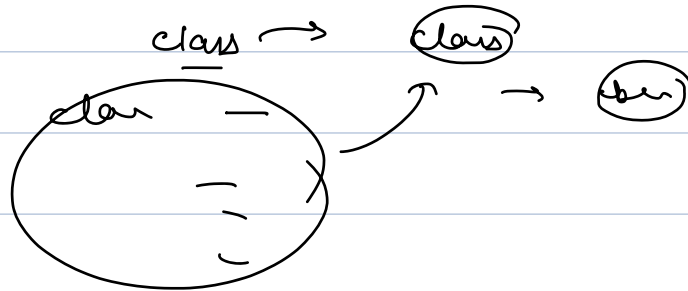
recovery $\rightarrow$ Timeout

③ Ignore it



Java Thread Lifecycle

volatile keyword,

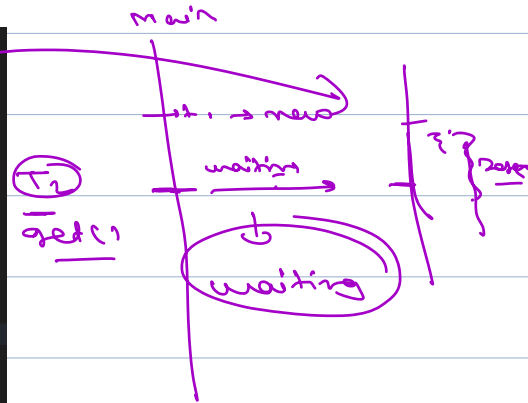


```

public class Test2 { new *
    public static void main(String[] args) { new *
        Thread t1 = new Thread() { new *
            @Override new *
            public void run() {
                try {
                    System.out.println("Print");
                    Thread.sleep(10000);
                } catch (InterruptedException e) {
                    throw new RuntimeException(e);
                }
            }
        };

        System.out.println(t1.getState().name());
        t1.start();
        try {
            Thread.sleep(10000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        System.out.println(t1.getState().name());
    }
}

```



```

public class Test { new *
    public static void main(String[] args) { new *
        Thread t1 = new Thread() {
            try {
                System.out.println("Print");
                Thread.sleep(20000);
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }
        };

        System.out.println(t1.getState().name());
        t1.start();
        System.out.println(t1.getState().name());
    }
}

```

